1. **Simulação do Insertion Sort**

Vetor inicial: [3, 1, 2]

Primeira passagem (i = 1)

- Chave = 1
- j = 0
- Comparação: 3 > 1 (verdadeiro)
- Deslocamento: o elemento 3 é movido para a posição 1
- Vetor temporário: [3, 3, 2]
- Atualização: j = -1
- Saída do laço interno: j < 0.
- Inserção da chave na posição 0.

Vetor após a primeira passagem: [1, 3, 2]

Contagem da passagem: 1 comparação entre elementos e 1 deslocamento

Segunda passagem (i = 2)

- Vetor inicial da passagem: [1, 3, 2]
- Chave = 2
- j = 1
- Primeira comparação: 3 > 2 (verdadeiro).
- Deslocamento: o elemento 3 é movido da posição 1 para a posição 2
- Vetor temporário: [1, 3, 3]
- Atualização: j = 0
- Segunda comparação: 1 > 2 (falso).
- Saída do laço interno: a[j] <= chave.
- Inserção da chave 2 na posição j + 1 = 1.

Vetor após a segunda passagem: [1, 2, 3]

Contagem da passagem: 2 comparações entre elementos e 1 deslocamento.

Passagem	Chave	Vetor após a passagem	Comparações	Deslocamentos
1	1	[1, 3, 2]	1	1
2	2	[1, 2, 3]	2	1

Totais:

- Passagens do laço externo: 2
- Comparações entre elementos: 3
- Deslocamentos do laço interno: 2
- Inserções da chave: 2

Formas de saída do laço interno:

1. j < 0: ocorre na primeira passagem, quando a chave chega ao início do vetor

2. a[j] <= chave: ocorre na segunda passagem, quando encontramos um elemento menor ou igual à chave

O Insertion Sort ordenou corretamente o vetor [3, 1, 2], produzindo [1, 2, 3]. A simulação exercitou as duas formas de saída do laço interno.

2. Simulação do Merge Sort

Vetor inicial: [3, 1, 2]

Primeira divisão:

- Tamanho do vetor = 3
- Meio = 3 / 2 = 1 (divisão inteira)
- Metade esquerda = [3]
- Metade direita = [1, 2]

Segunda divisão:

- O vetor [3] possui apenas um elemento, atingindo o caso-base.
- O vetor [1, 2] possui dois elementos e precisa ser dividido novamente.
- Meio = 2 / 2 = 1
- Metade esquerda = [1]
- Metade direita = [2]

Resultado das divisões:

[3, 1, 2] → [3] e [1, 2]

[1, 2] → [1] e [2]

Caso base: A recursão termina quando o vetor possui zero ou um elemento, pois ele já está ordenado e não precisa ser dividido novamente.

2.1. Intercalações do Merge Sort

Primeira intercalação: [1] e [2]

- Índices iniciais: i = 0, j = 0.
- Comparação: 1 <= 2 (verdadeiro).
- O elemento 1 é adicionado ao resultado.
- i = 1. A metade esquerda foi esgotada.
- O elemento 2 é copiado pelo laço de restante da direita.

Resultado: [1, 2]

Contagens:

- Comparações entre valores: 1
- Iterações do laço principal: 1
- Iterações do restante da esquerda: 0
- Iterações do restante da direita: 1

Segunda intercalação: [3] e [1, 2]

- Índices iniciais: i = 0, j = 0.
- Primeira comparação: 3 <= 1 (falso).
- O elemento 1 é adicionado ao resultado.
- j = 1.
- Segunda comparação: 3 <= 2 (falso).
- O elemento 2 é adicionado ao resultado.
- j = 2. A metade direita foi esgotada.
- O elemento 3 é copiado pelo laço de restante da esquerda.

Resultado: [1, 2, 3]

Contagens:

- Comparações entre valores: 2
- Iterações do laço principal: 2
- Iterações do restante da esquerda: 1
- Iterações do restante da direita: 0

2.2. Contagens totais da simulação

Operação	Quantidade
Chamadas totais de MERGE	5
Chamadas recursivas, sem contar a inicial	4
Casos-base atingidos	3
Intercalações realizadas	2
Comparações entre valores	3
Iterações do laço principal	3
Iterações do restante da esquerda	1
Iterações do restante da direita	1

2.3. Conclusão da simulação

O Merge Sort dividiu recursivamente o vetor [3, 1, 2] até chegar a vetores com um elemento. Depois, realizou duas intercalações, produzindo o resultado final [1, 2, 3].

Foram realizadas 5 chamadas de MERGE, 3 comparações entre valores e 5 iterações no total dos três laços de intercalação.

2.4. Simulação do Merge Sort - Vetor de tamanho par

Vetor inicial: [4, 2, 3, 1]

Divisões recursivas:

1. [4, 2, 3, 1] → [4, 2] e [3, 1]
2. [4, 2] → [4] e [2]
3. [3, 1] → [3] e [1]

Os vetores [4], [2], [3] e [1] atingem o caso-base por possuírem apenas um elemento.

Primeira intercalação: [4] e [2]

- Comparação: 4 <= 2 (falso).
- O elemento 2 é adicionado ao resultado.
- O elemento 4 é copiado pelo laço de restante da esquerda.
- Resultado: [2, 4].
- Contagens: 1 comparação, 1 iteração do laço principal e 1 do restante esquerdo.

Segunda intercalação: [3] e [1]

- Comparação: 3 <= 1 (falso).
- O elemento 1 é adicionado ao resultado.
- O elemento 3 é copiado pelo laço de restante da esquerda.
- Resultado: [1, 3].
- Contagens: 1 comparação, 1 iteração do laço principal e 1 do restante esquerdo.

Terceira intercalação: [2, 4] e [1, 3]

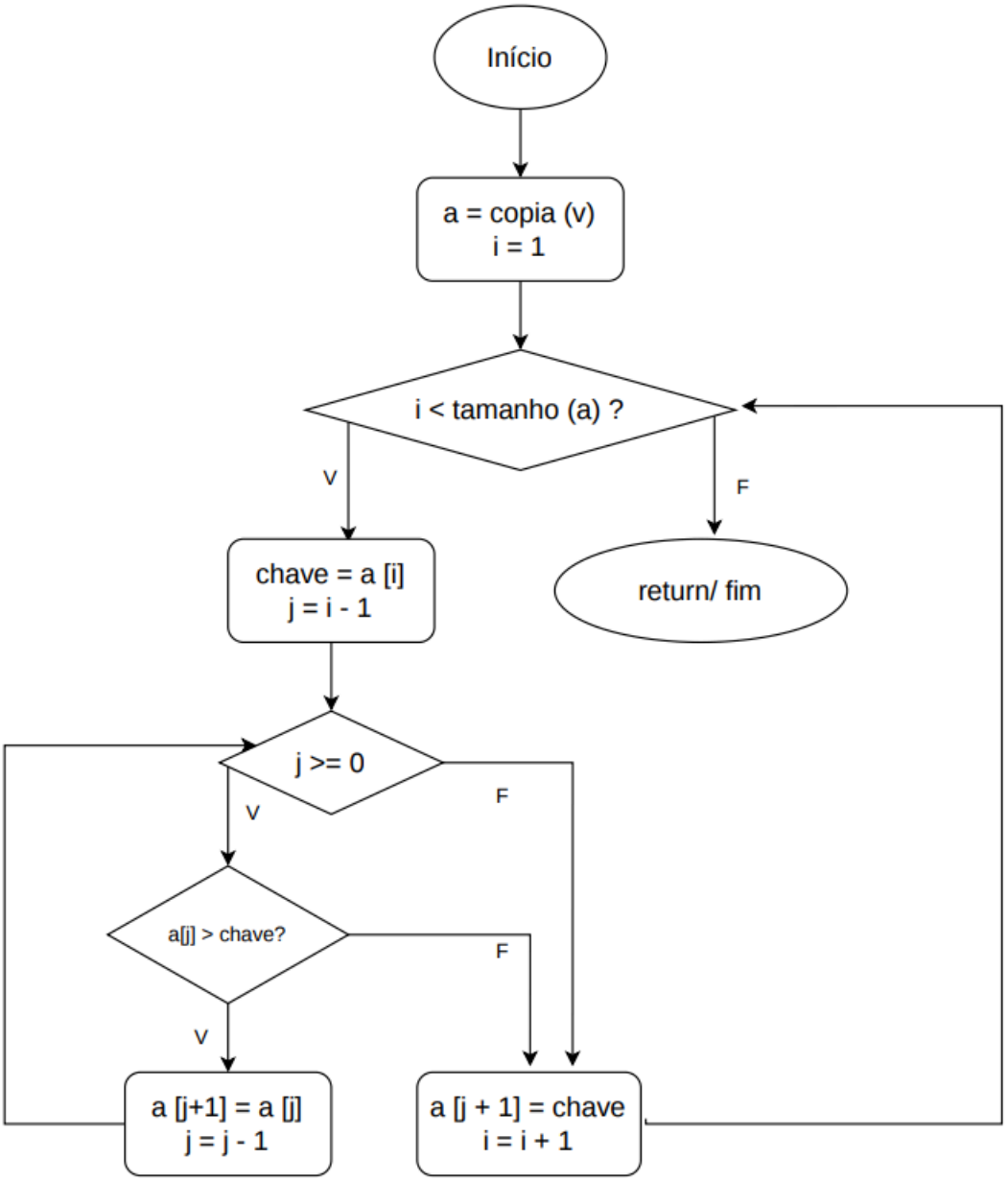
- Comparação: 2 <= 1 (falso). Adiciona 1.
- Comparação: 2 <= 3 (verdadeiro). Adiciona 2.
- Comparação: 4 <= 3 (falso). Adiciona 3.
- A metade direita foi esgotada.
- O elemento 4 é copiado pelo laço de restante da esquerda.
- Resultado: [1, 2, 3, 4].
- Contagens: 3 comparações, 3 iterações do laço principal e 1 do restante esquerdo.

Contagens totais:

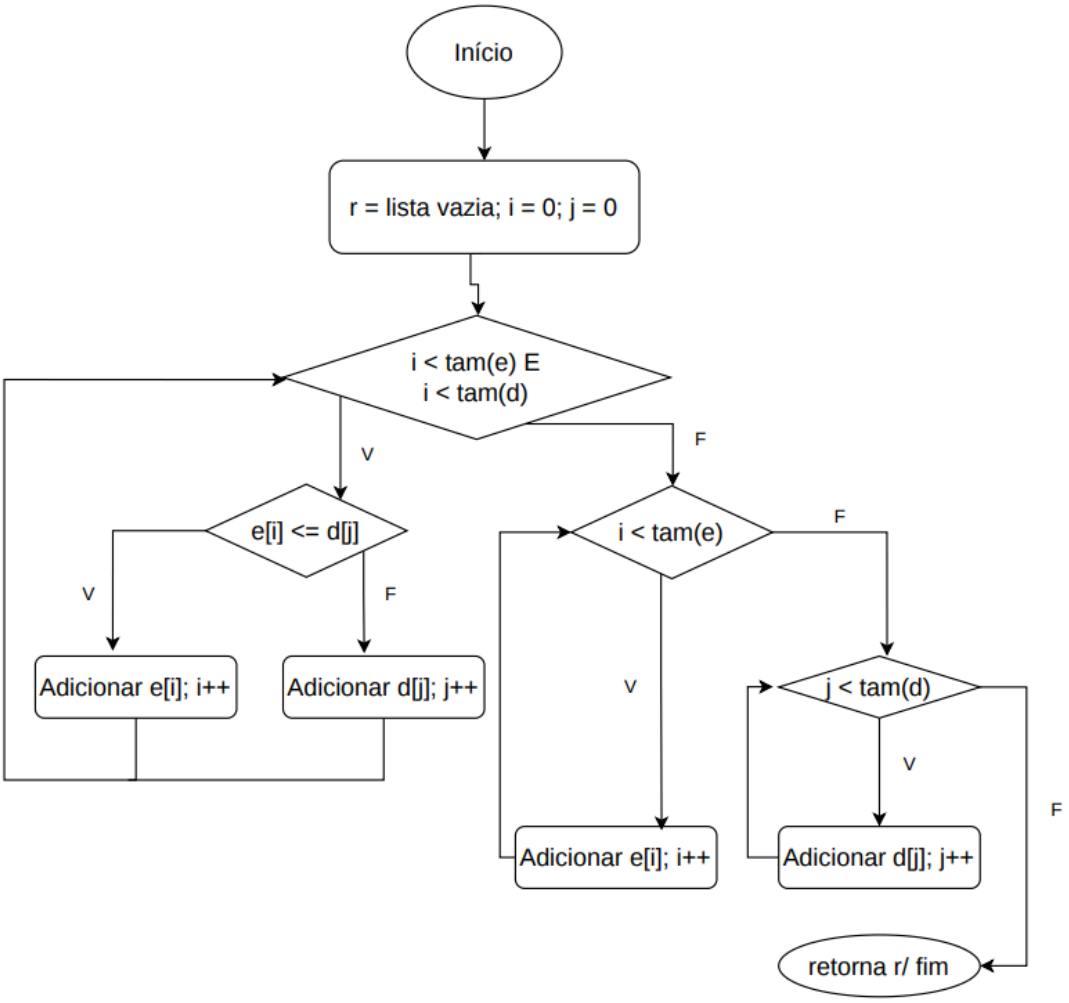
Operação	Quantidade

Chamadas de MERGE	7
Casos-base	4
Intercalações	3
Comparações entre valores	5
Laço principal	5
Restante da esquerda	3
Restante da direita	0
Total de iterações	8

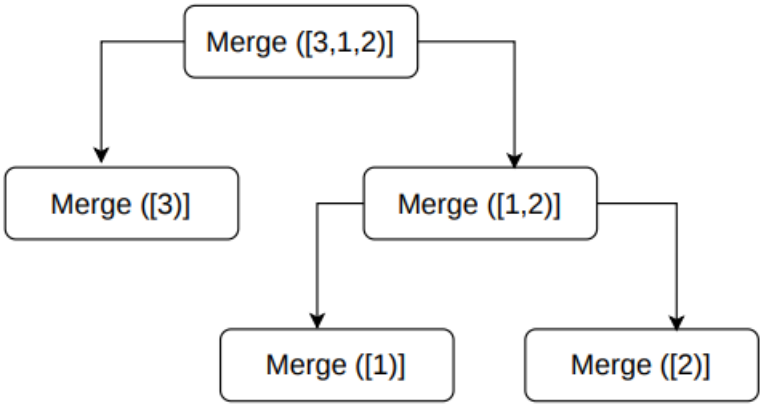
3. **Grafo de Fluxo de Controle - Insertion Sort**



3.1. Grafo de Fluxo de Controle - INTERCALAR (Merge Sort)



3.2. Árvore de Chamadas Recursivas - Merge Sort



4. Planejamento dos Casos de Teste

4.1. Entradas e resultados esperados

ID	Entrada	Saída esperada	Objetivo
T01	[ ]	[ ]	Vetor vazio
T02	[5]	[5]	Um elemento
T03	[1,2]	[1,2]	Dois elementos ordenados
T04	[2,1]	[1,2]	Dois elementos invertidos
T05	[1,2,3,4]	[1,2,3,4]	Vetor crescente
T06	[4,3,2,1]	[1,2,3,4]	Vetor decrescente
T07	[2,2,2]	[2,2,2]	Valores repetidos
T08	[3,-1,3,0]	[-1,0,3,3]	Negativos, zero e repetidos
T09	[3,1,2]	[1,2,3]	Tamanho ímpar e deslocamentos
T10	null	IllegalArgumentException	Entrada inválida

4.2. Contagens previstas — Insertion Sort

ID	Passagens externas	Comparações entre valores	Deslocamentos
T01	0	0	0
T02	0	0	0
T03	1	1	0
T04	1	1	1
T05	3	3	0
T06	3	6	6
T07	2	2	0
T08	3	5	3
T09	2	3	2
T10	0	0	0

4.3. Caminhos previstos — Insertion Sort

ID	Caminho previsto
T01 e T02	Laço externo não executado
T03	Laço interno sem deslocamentos
T04	Um deslocamento e saída por j < 0
T05	Passagens externas sem deslocamentos
T06	Vários deslocamentos e saída por j < 0
T07	Comparações com valores iguais
T08	Deslocamentos e saída por a[j] <= chave
T09	Saídas por j < 0 e a[j] <= chave
T10	Rejeição por entrada nula

4.4. Contagens previstas — Merge Sort

ID	Chamadas MERGE	Intercalações	Comparações	Laço principal	Restante Esquerda	Restante Direita
T01	1	0	0	0	0	0
T02	1	0	0	0	0	0
T03	3	1	1	1	0	1
T04	3	1	1	1	1	0
T05	7	3	4	4	0	4
T06	7	3	4	4	4	0
T07	5	2	2	2	0	3
T08	7	3	5	5	2	1
T09	5	2	3	3	1	1
T10	0	0	0	0	0	0

4.5. Caminhos previstos — Merge Sort

ID	Caminho previsto
T01	Caso base com vetor vazio
T02	Caso base com um elemento
T03	Uma divisão, esquerda esgota primeiro
T04	Uma divisão, direita esgota primeiro
T05	Divisões recursivas e restante direito
T06	Divisões recursivas e restante esquerdo
T07	Comparações com empate
T08	Recursão, empate e ambos os laços de restante
T09	Divisão ímpar e ambos os laços de restante
T10	Exceção antes da recursão

4.6. Testes Específicos do Procedimento INTERCALAR

ID	Esquerda	Direita	Resultado esperado
I01	[ ]	[1, 3]	[1, 3]
I02	[1, 3]	[ ]	[1, 3]
I03	[1]	[3]	[1, 3]
I04	[3]	[1]	[1, 3]
I05	[2, 2]	[2, 3]	[2, 2, 2, 3]

Verificação do vetor original

Além de verificar se os resultados estão em ordem crescente, os testes também devem confirmar que o Insertion Sort e o Merge Sort não modificam o vetor original. Os dois algoritmos devem retornar um novo vetor ordenado, mantendo os valores e a ordem do vetor de entrada.

5. Revisão do Plano de Testes

O plano de testes foi elaborado considerando diferentes situações, como vetores vazios, ordenados, invertidos, com valores repetidos, negativos e tamanhos pares e ímpares.

Durante a revisão com outra dupla, deverão ser verificados os resultados esperados, os caminhos dos algoritmos e as contagens previstas, buscando identificar possíveis erros ou casos que não foram considerados.

Após essa revisão, serão registrados os ajustes necessários antes da implementação e execução dos testes na Parte 2.